



Manual de Integração

API de Averbação AverbGo

Versão	2.0
Data	Julho de 2026
Público	Equipes de desenvolvimento de TMS, ERP e emissores fiscais
Ambientes	api.averbgo.com.br · api.qa.averbgo.com.br

Sumário

1 Visão geral

Para quem é este manual
Como funciona, em cinco passos

Documentos aceitos
O que mudou nesta versão do manual

2 Ambientes

Homologação

3 Autenticação

Onde obter a chave
A chave já nasce pronta

Chave inválida ou desativada

4 Requisição

Headers
Corpo da requisição
Exemplo — cURL
Exemplo — C#

Exemplo — Java
Exemplo — PHP
Quando enviar
Resposta

5 Tags extras

Tags disponíveis
Exemplo completo

Quais tags são obrigatórias?

6 Visão geral dos retornos

Os seis blocos
Árvore de decisão
Como determinar o resultado final

Onde está o número da averbação
Campos sempre presentes

7 Multi-hierarquia

Estrutura
Campos de cada hierarquia
Consolidação do resultado

O ANTT vem por hierarquia
Nem toda hierarquia tem endorsement

8 Produtos

Produtos existentes
Campos de cada produto
Códigos observados em statusCode
Conteúdo de data por produto

Consolidação
Exemplo — falha parcial
Exemplo — reenvio de documento já averbado

9 Averbação (endorsement)

Exemplo

Campos principais

10 MDF-e

Exemplo

Campos

Cancelamento e encerramento

11 Eventos

Cancelamento de CT-e / NF-e / Outros (event)

Cancelamento e encerramento de MDF-e (event_mdfe)

Pré-requisito

Evento repetido

12 Erros

Códigos HTTP

Formatos de corpo em erro

Leitura recomendada da mensagem

Códigos de error conhecidos

Mensagens frequentes

O que reenviar

13 Guia de implementação

Tratamento completo da resposta

Regras de ouro

Fila de envio

Retentativa

Ordem dos envios

O que exibir ao usuário

Avisos consolidados

Checklist de homologação

14 Exemplos por cenário

1. Averbação de CT-e — sucesso

2. Emitente com dois produtos — tudo processado

3. Sucesso parcial

4. Multi-hierarquia — averbação

5. Multi-hierarquia — cancelamento com falha parcial

6. MDF-e processada

7. Encerramento de MDF-e

8. Recusa por prazo

9. Documento já averbado

10. Chave de emitente inativo

15 Suporte e FAQ

Contato

Perguntas frequentes

Histórico deste manual

1 Manual de Integração — API AverbGo

Este manual descreve como integrar um sistema de gestão de transporte (TMS), ERP ou emissor de documentos fiscais à **API de averbação do AverbGo**.

A integração tem um único objetivo: **enviar à seguradora, em tempo real, os documentos fiscais emitidos pelo segurado**, recebendo de volta o número de averbação (ANTT) e a situação de cada produto contratado.

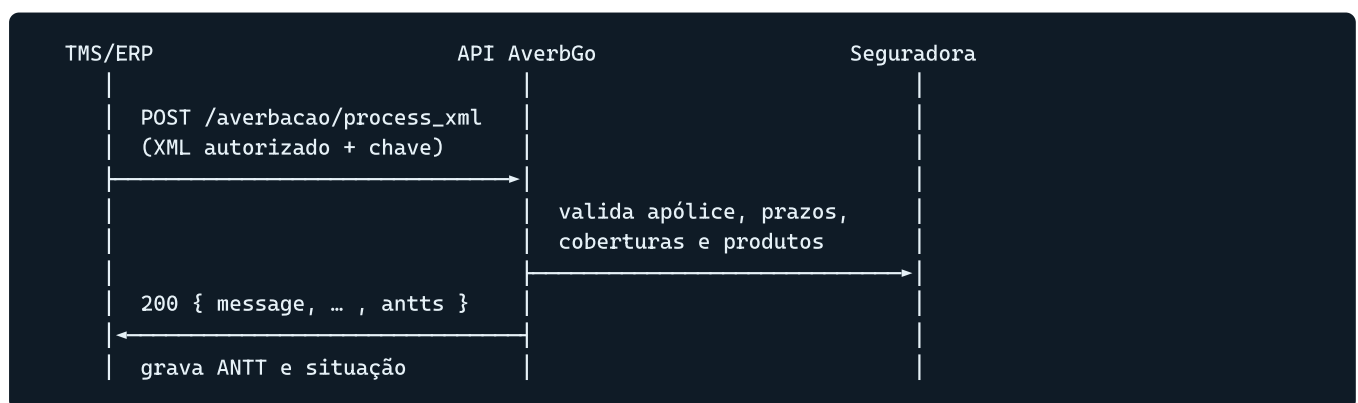
Para quem é este manual

Times de desenvolvimento que precisam:

- enviar CT-e, NF-e, MDF-e e documentos "Outros" logo após a autorização na SEFAZ;
- enviar eventos de **cancelamento** e **encerramento**;
- interpretar corretamente os diferentes formatos de resposta;
- tratar erros e recusas de averbação sem perder documentos.

Como funciona, em cinco passos

1. O segurado gera a **chave de acesso** no portal AverbGo.
2. O TMS emite o documento fiscal e obtém o XML **autorizado e protocolado** pela SEFAZ.
3. O TMS envia o XML para `POST /averbacao/process_xml`, com a chave no header `Authorization`.
4. A API processa o documento em todos os **produtos** contratados pelo emitente (Transporte, RC-V, Transmissão) e devolve um JSON com o resultado de cada um.
5. O TMS registra o retorno — em especial o número **ANTT**, que é o número da averbação — e trata eventuais falhas.



Documentos aceitos

Documento	Modelo	Tag inicial esperada no XML
CT-e	57	<cteProc> / <CTe>
NF-e	55	<nfeProc> / <NFe>
MDF-e	58	<mdfeProc> / <MDFe>
Outros documentos de transporte	91 a 99 e 1001 a 1009	conforme o layout do documento
Evento de cancelamento de CT-e	110111	<procEventoCTe> / <eventoCTe>
Evento de cancelamento de NF-e	110111	<procEventoNFe>
Evento de cancelamento de MDF-e	110111	<procEventoMDFe> / <eventoMDFe>
Evento de encerramento de MDF-e	110112	<procEventoMDFe>

Informação

O XML deve estar no padrão SEFAZ e **protocolado** (documento autorizado). XMLs sem protocolo de autorização são recusados.

Precisa enviar outro tipo de documento?

Consulte antes o suporte AverbGo: sac@averbgo.com.br.

O que mudou nesta versão do manual

Esta edição substitui integralmente a documentação anterior, que descrevia **apenas um** formato de resposta (`endorsement`). A API evoluiu e hoje responde em **seis formatos distintos**, conforme o tipo de documento e a quantidade de emitentes/produtos envolvidos:

Formato	Quando ocorre	Capítulo
<code>emitentes[]</code>	mesmo emitente vinculado a mais de uma hierarquia (cliente/parceiro)	Multi-hierarquia
<code>produtos{}</code>	emitente com mais de um produto contratado	Produtos
<code>endorsement</code>	averbação de CT-e, NF-e e Outros	Averbação
<code>mdfe</code>	processamento de MDF-e	MDF-e
<code>event</code>	evento de cancelamento de CT-e/NF-e	Eventos
<code>event_mdfe</code>	cancelamento/encerramento de MDF-e	Eventos

Leia antes de codificar

Um integrador que trate **apenas** `endorsement` vai considerar como falha documentos que foram averbados com sucesso em outros formatos. O capítulo [Visão geral dos retornos](#) traz a árvore de decisão completa.

2 Ambientes

O AverbGo disponibiliza dois ambientes:

Ambiente	URL base	Uso
Produção (PRD)	<code>https://api.averbgo.com.br/v1</code>	Documentos reais, averbações válidas
Qualidade (QA)	<code>https://api.qa.averbgo.com.br</code>	Homologação da integração

Montagem da URL final:

```
PRD → https://api.averbgo.com.br/v1/averbacao/process_xml
QA → https://api.qa.averbgo.com.br/averbacao/process_xml
```

Atenção ao prefixo `/v1`

O prefixo `/v1` existe **apenas em produção**. O ambiente de qualidade responde na raiz. Monte a URL a partir da constante de ambiente, nunca concatenando `/v1` fixo no código.

Homologação

Antes de liberar a integração em produção, valide em QA no mínimo:

- envio de CT-e, NF-e e MDF-e autorizados;
- envio de evento de cancelamento e, para MDF-e, de encerramento;
- um cenário de **recusa** (por exemplo, documento fora do prazo de averbação);
- um cenário de **duplicidade** (reenvio do mesmo documento);
- um cenário de **falha parcial**, quando o emitente tem mais de um produto contratado.

As chaves de acesso de QA são independentes das de produção e devem ser geradas no portal correspondente.

Informação

Os dados enviados em QA não geram averbação válida e não são considerados pela seguradora.

3 Autenticação

Toda requisição é autenticada por uma **chave de acesso** (`SECRET_KEY`) enviada no header `Authorization` .

```
Authorization: SECRET_KEY
```

Sem `Bearer`

A chave vai **pura** no header, sem o prefixo `Bearer` e sem aspas.

Onde obter a chave

O próprio segurado gera a chave no portal AverbGo, em <https://averbgo.com.br/main/chaves>.

Cada chave está vinculada a um emitente. Um TMS que atende vários embarcadores deve armazenar **uma chave por cliente** e usar a chave correta em cada envio — é a chave que identifica de quem é o documento.

Tratamento da chave

- Nunca versiona a chave no código-fonte nem a exponha no front-end.
- Armazene-a criptografada, com acesso restrito.
- Em caso de suspeita de vazamento, gere uma nova no portal e desative a anterior.

A chave já nasce pronta

Não há etapa de validação ou ativação: assim que é gerada no portal, a chave está apta a enviar documentos. Basta configurá-la no TMS e começar a integrar.

Se quiser confirmar que a chave está correta antes de colocar a operação no ar, o caminho é enviar um documento no ambiente de [qualidade](#) e verificar o retorno.

Chave inválida ou desativada

Uma chave revogada, expirada ou de emitente inativo faz a API responder **HTTP 401**.

Pare o envio no 401

`401` não é erro transitório: reenviar não resolve. O comportamento recomendado é **interromper a fila daquele cliente**, sinalizar a situação ao usuário e orientá-lo a verificar a chave no portal AverbGo. Continuar tentando apenas acumula documentos em erro.

4 Requisição

Um único endpoint recebe **todos** os tipos de documento e de evento.

```
POST /averbacao/process_xml
```

Ambiente	URL completa
Produção	<code>https://api.averbgo.com.br/v1/averbacao/process_xml</code>
Qualidade	<code>https://api.qa.averbgo.com.br/averbacao/process_xml</code>

Headers

```
Authorization: SECRET_KEY
Content-Type: application/xml
```

Header	Obrigatório	Observação
<code>Authorization</code>	sim	Chave de acesso do emitente, gerada em averbgo.com.br/main/chaves . Sem prefixo <code>Bearer</code> .
<code>Content-Type</code>	sim	Sempre <code>application/xml</code> .

Corpo da requisição

O corpo é o **conteúdo do XML**, enviado como texto puro — não use `multipart/form-data`, JSON envelopado nem Base64.

- Codificação **UTF-8**.
- XML **autorizado e protocolado** pela SEFAZ.
- Um documento por requisição.

Exemplo — cURL

```
curl --request POST \
  --url https://api.averbgo.com.br/v1/averbacao/process_xml \
  --header 'Authorization: SECRET_KEY' \
  --header 'Content-Type: application/xml' \
  --data-binary '@CTe35260721976218000156570010000001851784420492-procCte.xml'
```

Exemplo — C#

```
using var client = new HttpClient();
client.DefaultRequestHeaders.Add("Authorization", secretKey);

var xml = await File.ReadAllTextAsync(caminhoXml, Encoding.UTF8);
var body = new StringContent(xml, Encoding.UTF8, "application/xml");

var resposta = await client.PostAsync(
    "https://api.averbgo.com.br/v1/averbacao/process_xml", body);

var json = await resposta.Content.ReadAsStringAsync();
```

Exemplo — Java

```
HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.averbgo.com.br/v1/averbacao/process_xml"))
    .header("Authorization", secretKey)
    .header("Content-Type", "application/xml")
    .POST(HttpRequest.BodyPublishers.ofFile(Path.of(caminhoXml)))
    .build();

HttpResponse<String> response =
    client.send(request, HttpResponse.BodyHandlers.ofString());
```

Exemplo — PHP

```
$ch = curl_init('https://api.averbgo.com.br/v1/averbacao/process_xml');
curl_setopt_array($ch, [
    CURLOPT_POST => true,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER => [
        'Authorization: ' . $secretKey,
        'Content-Type: application/xml',
    ],
    CURLOPT_POSTFIELDS => file_get_contents($caminhoXml),
]);
$json = curl_exec($ch);
```

Quando enviar

Envie **imediatamente após a autorização** do documento na SEFAZ. As apólices têm prazo de averbação (por exemplo, "antes do embarque" ou "N horas após a emissão"); documentos enviados fora desse prazo são recusados com `AVERBACAO_RECUSADA`.

Para eventos, envie o XML do evento assim que ele for autorizado — o documento original já deve ter sido enviado antes.

Ordem dos envios

O evento de cancelamento de um documento que **nunca foi enviado** à API é recusado. Garanta que o documento original tenha sido processado antes de enviar o evento.

Resposta

A resposta é sempre `application/json`. Os formatos possíveis estão descritos em [Retornos da API](#).

5 Tags extras

Alguns dados exigidos pela seguradora não têm campo próprio no layout da SEFAZ. Eles são enviados como **observações do contribuinte**, dentro da tag `<compl>` do documento:

```
<compl>
  <ObsCont xCampo="NOME_DA_TAG">
    <xTexto>VALOR</xTexto>
  </ObsCont>
</compl>
```

- O atributo `xCampo` recebe o **nome da tag extra** (respeitando maiúsculas e minúsculas).
- O valor vai em `<xTexto>`.
- Tags que aceitam múltiplos valores (motoristas, veículos) podem se repetir.

Limite de 10 ocorrências

A SEFAZ aceita no máximo **10 grupos** `ObsCont` **por documento**. Cada tag extra ocupa uma ocorrência — e tags repetidas contam individualmente: dois `cpfMotorista` e dois `idVeiculo` já consomem quatro das dez.

Envie apenas as tags exigidas pela apólice do segurado. Se o documento passar do limite, a própria SEFAZ rejeita a autorização, antes mesmo de o XML chegar ao AverbGo.

Tags disponíveis

Tag (CT-e e NF-e)	Campo	Formato / observação
<code>modal</code>	Tipo de modal	Código do modal
<code>ramo</code>	Ramo	Ramo de seguro
<code>ufCarrega</code>	UF de origem	Sigla (ex.: <code>SP</code>)
<code>ufDescarrega</code>	UF de destino	Sigla
<code>cidadeCarrega</code>	Cidade de origem	Nome ou código IBGE
<code>cidadeDescarrega</code>	Cidade de destino	Nome ou código IBGE
<code>tipoMerca</code>	Tipo de mercadoria	
<code>cpfMotorista</code>	CPF do motorista	Somente números; pode repetir
<code>rgMotorista</code>	RG do motorista	Pode repetir
<code>codLiberacao</code>	Código de liberação do motorista	
<code>idVeiculo</code>	Identificação do veículo	Placa; pode repetir
<code>veiculoProprio</code>	Transporte com veículo próprio	<code>S</code> ou <code>N</code>
<code>meiosProprios</code>	Transporte por meios próprios	<code>S</code> ou <code>N</code>
<code>rastreador</code>	Carga rastreada	<code>S</code> ou <code>N</code>
<code>escolta</code>	Carga escoltada	<code>S</code> ou <code>N</code>
<code>rcfdc</code>	Cobertura RCF-DC	<code>S</code> ou <code>N</code>
<code>opCargaDescarga</code>	Operação de carga e descarga	<code>S</code> ou <code>N</code>
<code>opIcamento</code>	Operação de içamento	<code>S</code> ou <code>N</code>
<code>opRemocao</code>	Operação de remoção	<code>S</code> ou <code>N</code>
<code>container</code>	Valor do container	Decimal com ponto (ex.: <code>1500.00</code>)
<code>acessorios</code>	Valor dos acessórios	Decimal
<code>frete</code>	Valor do frete	Decimal
<code>despesas</code>	Valor das despesas	Decimal
<code>impostos</code>	Valor dos impostos	Decimal — somente CT-e (na NF-e o dado é nativo)
<code>lucrosEsperados</code>	Valor dos lucros esperados	Decimal — somente NF-e
<code>avarias</code>	Valor de avarias	Decimal

Tag (CT-e e NF-e)	Campo	Formato / observação
embarque	Data e hora do embarque	ISO 8601 com fuso: 2022-11-28T23:45:59-03:00
tipViagemInternacional	Importação / exportação	
cnpjIsencao	CNPJ de isenção	Somente números

Valores monetários

Use ponto como separador decimal e não use separador de milhar: 1500.00, nunca 1.500,00.

Exemplo completo

```
<compl>
  <obsCont xCampo="embarque">
    <xTexto>2022-11-28T23:45:59-03:00</xTexto>
  </obsCont>
  <obsCont xCampo="cpfMotorista">
    <xTexto>36770479869</xTexto>
  </obsCont>
  <obsCont xCampo="cpfMotorista">
    <xTexto>00011122285</xTexto>
  </obsCont>
  <obsCont xCampo="idVeiculo">
    <xTexto>GEI6644</xTexto>
  </obsCont>
  <obsCont xCampo="idVeiculo">
    <xTexto>GEI6655</xTexto>
  </obsCont>
  <obsCont xCampo="rcfdc">
    <xTexto>S</xTexto>
  </obsCont>
  <obsCont xCampo="container">
    <xTexto>0.01</xTexto>
  </obsCont>
  <obsCont xCampo="acessorios">
    <xTexto>20.00</xTexto>
  </obsCont>
  <obsCont xCampo="despesas">
    <xTexto>50.00</xTexto>
  </obsCont>
  <obsCont xCampo="impostos">
    <xTexto>30.00</xTexto>
  </obsCont>
  <obsCont xCampo="avarias">
    <xTexto>40.00</xTexto>
  </obsCont>
</compl>
```

As tags fazem parte do XML assinado

As `obsCont` precisam estar no XML **antes da assinatura e da autorização**. Não é possível acrescentá-las depois: o documento perderia a validade da assinatura digital.

Quais tags são obrigatórias?

Depende da apólice e do ramo contratado pelo segurado. Em caso de dúvida sobre quais tags o cliente precisa enviar, consulte sac@averbgo.com.br.

6 Visão geral dos retornos

Toda resposta da API é um JSON com um campo `message` e, conforme o caso, **um** bloco de dados. O bloco presente indica o que foi processado.

```
{
  "message": "texto descritivo do processamento",
  "<bloco de dados>": { ... }
}
```

Os seis blocos

Bloco	Significado	Detalhamento
<code>emitentes</code>	Mesmo emitente vinculado a mais de uma hierarquia (cliente/parceiro). Cada hierarquia traz seus próprios produtos.	Multi-hierarquia
<code>produtos</code>	Resultado por produto contratado (Transporte, RC-V, Transmissão).	Produtos
<code>endorsement</code>	Averbação de CT-e, NF-e ou Outros. Contém o número ANTT .	Averbação
<code>mdfe</code>	Processamento de MDF-e.	MDF-e
<code>event</code>	Evento de cancelamento de CT-e / NF-e / Outros.	Eventos
<code>event_mdfe</code>	Cancelamento ou encerramento de MDF-e.	Eventos

Os blocos podem se combinar: é comum receber `message` + `endorsement` + `produtos`, ou `message` + `mdfe` + `produtos`.

Não assuma um formato fixo

O formato varia conforme o tipo de documento, a quantidade de emitentes envolvidos e os produtos contratados pelo segurado — e pode mudar para um mesmo cliente quando ele contrata um novo produto. Implemente a árvore de decisão abaixo, não um `if` para um único campo.

Árvore de decisão

Ordem recomendada de verificação:

1. HTTP 2xx ?
 não → trate como erro (ver "Erros")
 sim ↓
2. resposta.emitentes é um array ?
 sim → percorra as hierarquias e consolide o resultado [Multi-hierarquia]
 não ↓
3. resposta.produtos existe ?
 sim → consolide o resultado dos produtos [Produtos]
 não ↓
4. qual bloco veio?
 endorsement → averbação concluída, leia antts[] [Averbação]
 mdfe → MDF-e processada [MDF-e]
 event → evento de cancelamento processado [Eventos]
 event_mdfe → cancelamento/encerramento de MDF-e [Eventos]
 nenhum → leia message; trate como não averbado

Como determinar o resultado final

O sinal mais confiável **não** é o código HTTP isolado, e sim a combinação abaixo:

Situação	Como identificar	Resultado para o TMS
Sucesso total	HTTP 2xx e nenhum produto com <code>success: false</code>	Averbado
Sucesso parcial	HTTP 2xx e pelo menos um produto com <code>success: true</code> e outro com <code>success: false</code>	Averbado com pendência — exige tratamento
Falha	HTTP de erro, ou todos os produtos com <code>success: false</code>	Não averbado

Sucesso parcial existe e precisa de tratamento

Quando o emitente tem mais de um produto contratado, é possível que o Transporte seja averbado e o RC-V falhe (ou o contrário). A resposta ainda vem com HTTP de sucesso. Se o TMS considerar "HTTP 200 = tudo certo", o documento fica com um produto **não averbado** sem que ninguém perceba.

Onde está o número da averbação

O número **ANTT** é o número da averbação e deve ser gravado pelo TMS.

Formato do retorno	Caminho do ANTT
<code>endorsement</code>	<code>endorsement.antts[].antt</code> — pode haver mais de um (um por apólice)
<code>produtos</code>	<code>produtos.TRANSPORTE.data.endorsement.antts[].antt</code>
<code>emitentes</code>	<code>emitentes[].produtos.TRANSPORTE.data.endorsement.antts[].antt</code> — por hierarquia
<code>mdfe</code>	Não há ANTT. O identificador é <code>mdfe.id_averbgo_mdfe</code>
<code>event</code> / <code>event_mdfe</code>	Não há ANTT. São eventos sobre um documento já averbado

Mais de um ANTT

`antts` é um **array**. Emitente com duas apólices vigentes recebe um número por apólice. Grave todos: cada um se refere a uma apólice diferente (`numero_apolice`).

Campos sempre presentes

Campo	Tipo	Descrição
<code>message</code>	string	Descrição do processamento. Em falhas parciais, traz o resumo por produto separado por <code> </code> .

``message`` é texto livre

A `message` serve para exibição ao usuário e para log. **Não use a `message` como regra de negócio** — a redação pode mudar. Baseie a lógica em `success`, `statusCode` e `error`.

7 Retorno multi-hierarquia (`emitentes`)

Um mesmo emitente pode estar vinculado a **mais de uma hierarquia** — combinações distintas de cliente e parceiro (seguradora), cada uma com suas próprias apólices e produtos. Quando isso acontece, um único envio, com **uma única chave de acesso**, é processado em todas as hierarquias e a API responde com o array `emitentes`.

Cada item do array é **uma hierarquia**, não um emitente diferente: repare que o campo `emitente` costuma repetir o mesmo nome em todos os itens, variando `cliente` e `parceiro`.

Verifique este bloco primeiro

`emitentes` tem precedência sobre todos os demais. Se ele existir, os dados de averbação estão **dentro** dele — não procure `endorsement` na raiz.

Estrutura

```
{
  "message": "2 emitente(s) processado(s) com sucesso",
  "emitentes": [
    {
      "idPessoaEmitente": "f6133655-d982-4337-a7ad-51ac18c9c170",
      "emitente": "NR PARTICIPACOES",
      "cliente": "GPS-PAMCARY",
      "parceiro": "MAPFRE",
      "produtos": {
        "TRANSPORTE": {
          "success": true,
          "statusCode": 201,
          "message": "Processo de Averbação concluído com sucesso! - Cliente: GPS-PAMCARY - Parceiro: MAPFRE - Emitente: NR PARTICIPACOES",
          "data": {
            "endorsement": { "...": "ver capítulo Averbação" }
          }
        }
      }
    },
    {
      "idPessoaEmitente": "...",
      "emitente": "NR PARTICIPACOES",
      "cliente": "DATASYSTEM - RCV",
      "parceiro": "DATASYSTEM",
      "produtos": {
        "RCV": {
          "success": true,
          "statusCode": 202,
          "message": "Documento RC-V processado - Cliente: DATASYSTEM - RCV - Parceiro: DATASYSTEM - Emitente: NR PARTICIPACOES",
          "data": { "processed": true, "pending": true, "...": "..." }
        }
      }
    }
  ]
}
```

No exemplo acima é **o mesmo emitente** (NR PARTICIPACOES) em duas hierarquias: uma com o cliente GPS-PAMCARY e a seguradora MAPFRE, outra com o cliente e parceiro DATASYSTEM. Cada hierarquia processou o produto que lhe cabe.

Campos de cada hierarquia

Campo	Tipo	Descrição
<code>idPessoaEmitente</code>	string (UUID)	Identificador do emitente no AverbGo naquela hierarquia
<code>emitente</code>	string	Nome do emitente
<code>cliente</code>	string	Cliente (segurado) da hierarquia
<code>parceiro</code>	string	Parceiro/seguradora da hierarquia
<code>produtos</code>	objeto	Mapa <code>NOME_DO_PRODUTO</code> → resultado. Ver Produtos

Use os campos estruturados

`emitente`, `cliente` e `parceiro` já vêm prontos e identificam a hierarquia. Não é necessário extrair esses nomes do texto da `message` — que também os repete, mas em formato livre.

A `message` do topo fala em "emitentes"

A mensagem de resumo usa a redação `"2 emitente(s) processado(s) com sucesso"`, contando **hierarquias**, não emitentes distintos. É texto de exibição — a contagem confiável é `emitentes.length`.

Consolidação do resultado

Percorra **todos os produtos de todas as hierarquias**:

```
sucessos = produtos com success == true
falhas   = produtos com success == false

sucessos > 0 e falhas > 0 → SUCESSO PARCIAL
sucessos > 0 e falhas == 0 → AVERBADO
sucessos == 0              → FALHA
```

Exemplo de resposta com falha parcial:

```
{
  "message": "[NR PARTICIPACOES] TRANSPORTE: Evento de cancelamento processado com sucesso | [NR PARTICIPACOES] RCV: ERRO - O documento não pertence ao emitente",
  "emitentes": [
    { "produtos": { "TRANSPORTE": { "success": true, "statusCode": 200, "data": { "event": [ ... ] } } } },
    { "produtos": { "RCV": { "success": false, "statusCode": 500,
                           "error": "EVENT_PROCESSING_ERROR",
                           "message": "O documento não pertence ao emitente - Cliente: ..." } } } }
  ]
}
```

Formato da `message` em falha parcial

Repare no prefixo `[EMITENTE] PRODUTO:` e no marcador `ERRO -`. É um resumo textual para exibição — a informação confiável está em `emitentes[].produtos[].success`.

O ANTT vem por hierarquia

Cada hierarquia tem a **sua** apólice e, portanto, o seu número de averbação:

```
emitentes[i].produtos.TRANSPORTE.data.endorsement.antts[j].antt
```

Não leia apenas a primeira hierarquia

Um documento pode gerar ANTTs distintos em cada hierarquia, ou ter uma hierarquia averbada e outra recusada. Percorra o array inteiro e grave o resultado de cada uma.

Nem toda hierarquia tem `endorsement`

O conteúdo de `produtos[].data` **muda conforme a operação**:

Operação	Conteúdo de <code>data</code>
Averbação de CT-e / NF-e / Outros	<code>{ "endorsement": { ... } }</code>
Averbação de MDF-e	<code>{ "mdfe": { ... } }</code>
Cancelamento de CT-e / NF-e / Outros	<code>{ "event": [{ ... }] }</code>
Cancelamento / encerramento de MDF-e	<code>{ "event_mdfe": { ... }, "des_endereco_xml": "..." }</code>
Produto RC-V averbado	<code>{ "processed": true, "pending": true, "documento": { ... }, "policy": { ... } }</code>
Produto RC-V cancelado	um array , sem o envelope <code>event</code>
Evento já processado anteriormente	<code>{}</code> (objeto vazio)
Produto com falha	campo <code>data</code> ausente , com <code>error</code> preenchido

Sempre verifique a existência de `data`

Trate `data`, `data.endorsement`, `data.event` etc. como opcionais. Em cancelamentos e MDF-e **não existe** `data.endorsement`, e em falhas o `data` pode não vir. Acessos diretos sem verificação quebram a integração.

8 Retorno por produto (`produtos`)

Um mesmo documento pode ser processado em **vários produtos** contratados pelo emitente. Cada produto é avaliado de forma independente: um pode ser averbado e outro recusado.

O bloco `produtos` é um mapa cuja chave é o nome do produto:

```
{
  "message": "...",
  "produtos": {
    "TRANSPORTE": { "success": true, "statusCode": 201, "message": "...", "data": { "endorsement": { ...
  } } },
    "RCV": { "success": true, "statusCode": 202, "message": "...", "data": { ... } }
  }
}
```

Produtos existentes

Chave	Produto
<code>TRANSPORTE</code>	Seguro de transporte — é o produto que gera o ANTT
<code>RCV</code>	Responsabilidade Civil de Veículos (RC-V)
<code>TRANSMISSAO</code>	Transmissão do documento a terceiros

A lista pode crescer

Novos produtos podem ser adicionados. Trate `produtos` como um mapa dinâmico — percorra as chaves em vez de ler `produtos.TRANSPORTE` e `produtos.RCV` de forma fixa.

Campos de cada produto

Campo	Tipo	Obrigatório	Descrição
<code>success</code>	boolean	sim	Resultado do produto. É o campo que deve governar a lógica do TMS
<code>statusCode</code>	number	sim	Código detalhado do processamento daquele produto (ver tabela abaixo)
<code>message</code>	string	sim	Texto descritivo, geralmente com o sufixo - Cliente: ... - Parceiro: ... - Emitente: ...
<code>error</code>	string	não	Código do erro, quando <code>success: false</code> . Também aparece em alguns sucessos informativos (ex.: <code>DOCUMENTO_DUPLICADO</code>)
<code>data</code>	objeto/array	não	Dados do processamento. Formato varia por produto e operação

Códigos observados em `statusCode`

Código	Significado
200	Processado — documento ou evento já conhecido/atualizado
201	Averbado — registro criado
202	Aceito e em processamento (RC-V pendente de conclusão)
404	Recurso não encontrado (ex.: apólice inexistente)
422	Regra de negócio impediu a averbação (prazo, tipo de documento não autorizado)
500	Falha no processamento

``statusCode`` é do produto, não da requisição

Esse código é interno ao produto e **não** corresponde ao status HTTP da resposta. É comum receber HTTP 2xx com um produto em `statusCode: 500`.

Conteúdo de `data` por produto

TRANSPORTE

Operação	<code>data</code>
Averbação de CT-e / NF-e / Outros	<code>{ "endorsement": { ... } }</code> — ver Averbação
Averbação de MDF-e	<code>{ "mdfe": { ... } }</code> — ver MDF-e
Cancelamento de CT-e / NF-e	<code>{ "event": [{ ... }] }</code> — ver Eventos
Cancelamento / encerramento de MDF-e	<code>{ "event_mdfe": { ... }, "des_endereco_xml": "..." }</code>

RCV

Averbação:


```
{
  "success": true,
  "statusCode": 202,
  "message": "Documento RC-V processado",
  "data": {
    "processed": true,
    "pending": true,
    "documento": { "id_documento": "19295343-e2a7-4fce-b1a3-c27795aa8f54" },
    "policy": {
      "id_apolice": "b8c0ef17-35d5-45de-a6a2-3f37dcb67fd6",
      "cod_apolice": "221748",
      "cod_susep": 59,
      "des_amo": "Responsabilidade Civil de Veículos",
      "documento_tipo": "CTE",
      "vigencia": {
        "inicio": "2026-01-01T23:59:59.999-03:00",
        "fim": "2028-01-01T23:59:59.999-03:00"
      }
    }
  }
}
```

Campo	Descrição
<code>processed</code>	Documento processado no RC-V
<code>pending</code>	<code>true</code> indica conclusão assíncrona pendente
<code>documento.id_documento</code>	Identificador do documento no RC-V
<code>policy.cod_apolice</code>	Apólice utilizada
<code>policy.cod_susep</code>	Código SUSEP
<code>policy.des_amo</code>	Ramo (ex.: "Responsabilidade Civil de Veículos")
<code>policy.documento_tipo</code>	Tipo de documento aceito pela apólice
<code>policy.vigencia.inicio</code> / <code>fim</code>	Vigência da apólice

Cancelamento: `data` vem como **array** de eventos, sem envelope `event`.

Evento já processado antes: `data` vem como `{}`.

Consolidação

```
sucessos = produtos com success == true
falhas   = produtos com success == false
```

```
sucessos > 0 e falhas > 0 → SUCESSO PARCIAL (documento averbado em parte)
sucessos > 0 e falhas == 0 → AVERBADO
sucessos == 0              → FALHA
```

`produtos` vazio

Já foi observado o retorno `{"message": "Emitente sem meio de averbação habilitado para os produtos do documento", "produtos": {}}`. Um mapa vazio **não** significa sucesso — significa que nada foi processado. Trate `produtos` vazio como falha e exiba a `message` ao usuário.

Exemplo — falha parcial

```
{
  "message": "Falha ao criar registro de Veículo",
  "produtos": {
    "TRANSPORTE": {
      "success": false,
      "statusCode": 500,
      "message": "Falha ao criar registro de Veículo",
      "error": "MDFE_PROCESSING_ERROR"
    },
    "RCV": {
      "success": true,
      "statusCode": 202,
      "message": "Documento RC-V processado",
      "data": { "processed": true, "pending": true }
    }
  }
}
```

Resultado: **sucesso parcial** — o RC-V foi processado, o Transporte não. O documento precisa ser reenviado após a correção, ou tratado manualmente.

Exemplo — reenvio de documento já averbado

```
{
  "produtos": {
    "TRANSPORTE": { "success": true, "statusCode": 200, "error": "DOCUMENTO_DUPLICADO", "message":
    "..."},
    "RCV": { "success": true, "statusCode": 200, "error": "DOCUMENTO_DUPLICADO", "message":
    "..."}
  }
}
```

Duplicidade não é erro

`DOCUMENTO_DUPLICADO` vem com `success: true`: o documento **já estava averbado**. O reenvio é idempotente e não gera nova averbação — trate como sucesso.

9 Averbação (endorsement)

É o retorno da averbação de **CT-e, NF-e e documentos "Outros"**. Contém o número **ANTT** — o número da averbação.

Pode chegar de três formas:

<code>resposta.endorsement</code>	(documento simples)
<code>resposta.produtos.TRANSPORTE.data.endorsement</code>	(emitente com vários produtos)
<code>resposta.emitentes[i].produtos.TRANSPORTE.data.endorsement</code>	(multi-hierarquia)

Exemplo

```
{
  "message": "Processo de Averbação concluído com sucesso!",
  "endorsement": {
    "dfe": {
      "id_averbgo_dfe": "ec4c67bb-95f3-4118-a0d2-93c32a9fd851",
      "id_tipo_dfe": 55,
      "doc_apoio": "dfe-67744e6e-5ffd-4e18-8ecd-86b99771cbc8",
      "initialTag": "nfeproc",
      "docType": "nfe",
      "num_chave_dfe": "52260606314327000203550010104390271608635159",
      "xml": "XML ORIGINAL"
    },
    "id_documento": "ec4c67bb-95f3-4118-a0d2-93c32a9fd851",
    "cod_usuario": "JC DISTRIBUICAO",
    "data_emissao": "2026-06-01",
    "hora_emissao": "22:13:00",
    "valor_carga": 15.26,
    "valor_total": 15.26,
    "valor_carga_segurado": 15.26,
    "cidade_origem_nome": "APARECIDA DE GOIANIA - GO",
    "ufini": "GO",
    "cidade_destino_nome": "HIDROLINA - GO",
    "uffim": "GO",
    "cnpj_emitente": "06314327000203",
    "nom_fantasia_emitente": "JC DISTRIBUICAO",
    "cnpj_relacionamento": "06314327000114",
    "nom_fantasia_relacionamento": "JC DISTRIBUICAO",
    "motivo": "",
    "dhr_averbacao": "2026-06-02T11:28:59-03:00",
    "data_embarque": "2026-06-02",
    "hora_embarque": "11:28:56",
    "numero_documento": "10439027",
    "numserie_documento": "1",
    "tip_status": 1,
    "des_tipo_status": "Averbado",
    "tipo_documento": 55,
    "modal": "01",
    "urbano": "Sim",
    "isencao": { "id_isencao": null, "cod_docum_res": null, "tipo_isencao": "Sem isenção" },
    "apolices": [ { "id": "..." } ],
    "antts": [ { "antt": "0623801243318365800013555000001822047097", "numero_apolice": "33333" } ]
  }
}
```

Campos principais

Identificação

Campo	Tipo	Descrição
<code>id_documento</code>	string (UUID)	Identificador da averbação no AverbGo. Use-o em consultas ao suporte
<code>numero_documento</code>	string	Número do documento fiscal
<code>numSerie_documento</code>	string	Série do documento
<code>tipo_documento</code>	number string	Modelo do documento: <code>55</code> NF-e, <code>57</code> CT-e, <code>58</code> MDF-e. Documentos "Outros" usam os modelos <code>91</code> a <code>99</code> e <code>1001</code> a <code>1009</code>
<code>modal</code>	string	Código do modal (<code>01</code> rodoviário)
<code>urbano</code>	string	<code>Sim</code> / <code>Não</code>
<code>cod_usuario</code>	string	Usuário/identificação do emitente no AverbGo

`tipo_documento` ora número, ora texto`

Este campo já foi observado como número (`57`) e como string (`"57"`) em respostas diferentes. Faça a comparação convertendo para texto — por exemplo `String(tipo_documento) === "57"` — em vez de comparação estrita de tipo.

Situação

Campo	Tipo	Descrição
<code>tip_status</code>	number	<code>1</code> = averbado
<code>des_tipo_status</code>	string	Descrição da situação (ex.: <code>Averbado</code>)
<code>motivo</code>	string	Motivo, quando houver (recusas e situações especiais). Vazio em averbações normais
<code>dhr_averbacao</code>	string ISO 8601	Data/hora da averbação, com offset (<code>2026-06-02T11:28:59-03:00</code>)
<code>id_meio_averbacao</code>	number	Meio pelo qual a averbação foi feita

Fuso horário

`dhr_averbacao` traz o offset explícito. Converta a partir do offset recebido — não presuma UTC nem horário local, e não aplique correções fixas de fuso.

Números de averbação

Campo	Tipo	Descrição
<code>antts</code>	array	Lista de averbações geradas
<code>antts[].antt</code>	string	Número da averbação (ANTT) — grave este valor
<code>antts[].numero_apolice</code>	string	Apólice que gerou aquele número

Datas e valores

Campo	Tipo	Descrição
<code>data_emissao</code> / <code>hora_emissao</code>	string	Emissão do documento
<code>data_embarque</code> / <code>hora_embarque</code>	string	Embarque considerado
<code>valor_carga</code>	number	Valor da carga
<code>valor_total</code>	number	Valor total do documento
<code>valor_carga_segurado</code>	number	Valor efetivamente segurado
<code>cod_impor_ept</code>	number	Indicador de importação/exportação

Origem e destino

Campo	Tipo	Descrição
<code>cidade_origem_id</code> / <code>cidade_origem_nome</code>	string	Código IBGE e nome da cidade de origem
<code>ufini</code>	string	UF de origem
<code>cidade_destino_id</code> / <code>cidade_destino_nome</code>	string	Código IBGE e nome da cidade de destino
<code>uffim</code>	string	UF de destino

Emitente e relacionamento

Campo	Tipo	Descrição
<code>id_pessoa_emitente</code>	string (UUID)	Emitente no AverbGo
<code>cnpj_emitente</code>	string	CNPJ do emitente
<code>nom_fantasia_emitente</code>	string	Nome fantasia do emitente
<code>cnpj_relacionamento</code>	string	CNPJ do relacionamento (segurado/contratante)
<code>nom_fantasia_relacionamento</code>	string	Nome fantasia do relacionamento

Documento eletrônico (`dfe`)

Campo	Tipo	Descrição
<code>dfe.id_averbgo_dfe</code>	string (UUID)	Identificador do XML recebido
<code>dfe.num_chave_dfe</code>	string (44)	Chave de acesso do documento
<code>dfe.id_tipo_dfe</code>	number string	Modelo do documento
<code>dfe.docType</code>	string	Tipo detectado (<code>nfe</code> , <code>cte</code> , <code>mdfe</code>)
<code>dfe.initialTag</code>	string	Tag raiz identificada no XML
<code>dfe.doc_apoio</code>	string	Referência interna do documento de apoio
<code>dfe.xml</code>	string	Eco do XML enviado

``dfe.xml`` deixa a resposta grande

O XML original é devolvido dentro da resposta e, em alguns casos, também a árvore do documento já convertida em JSON. Uma resposta pode passar de 100 KB. Dimensione timeouts e o armazenamento de logs considerando isso — ou descarte `dfe.xml` antes de persistir.

Isenção

Campo	Tipo	Descrição
<code>isencao.tipo_isencao</code>	string	Ex.: <code>Sem isenção</code>
<code>isencao.id_isencao</code>	string null	Identificador da isenção
<code>isencao.cod_docum_res</code>	string null	Documento responsável

Apólices (`apolices[]`)

Lista das apólices consideradas. Campos mais usados:

Campo	Tipo	Descrição
<code>id_apolice</code>	string (UUID)	Identificador da apólice
<code>cod_apolice</code>	string	Número da apólice
<code>cod_externo</code>	string	Código externo/da seguradora
<code>cod_susep</code>	number	Código SUSEP
<code>id_amo</code> / <code>cod_sigla_amo</code> / <code>des_amo</code>	number/string	Ramo (ex.: <code>TRNAC</code> — Transporte Nacional)
<code>dhr_inicio_vigencia</code> / <code>dhr_fim_vigencia</code>	string ISO	Vigência
<code>sta_cobertura_roubo</code>	boolean	Cobertura de roubo
<code>sta_cobertura_2risco</code>	boolean	Cobertura de segundo risco
<code>vlr_img</code>	number	Limite máximo de garantia
<code>des_prazo averbacao</code> / <code>num_prazo averbacao</code>	string/number	Prazo de averbação (ex.: "Horas após Embarque")
<code>des_extrapolacao</code>	string	Comportamento ao extrapolar o limite (ex.: <code>Recusa</code>)
<code>tip_documento_autorizado</code>	array	Tipos de documento aceitos pela apólice
<code>nom_fantasia_emitente</code> / <code>nom_fantasia_relacionamento</code>	string	Partes da apólice

Campos adicionais

O bloco pode trazer campos extras conforme o ramo e o tipo de documento (por exemplo `tpServ`, `cnpj_tomador`, `condutores`, `veiculos`). Ignore com segurança os campos que o seu sistema não usa — e **não** falhe o parse ao encontrar campos desconhecidos.

10 MDF-e (mdfe)

Retorno do processamento de um **Manifesto Eletrônico de Documentos Fiscais**. Pode chegar como:

<code>resposta.mdfc</code>	(documento simples)
<code>resposta.produtos.TRANSPORTE.data.mdfc</code>	(emitente com vários produtos)
<code>resposta.emitentes[i].produtos.TRANSPORTE.data.mdfc</code>	(multi-hierarquia)

MDF-e não gera ANTT

O MDF-e é a viagem, não a averbação da carga. Não há `antt` neste retorno — o identificador do processamento é `id_averbgo_mdfc`. As averbações são as dos CT-e/NF-e relacionados.

Exemplo

```
{
  "message": "Processamento de MDFe concluido com sucesso!",
  "mdfe": {
    "id_verbgo_mdfe": "1075df34-b408-4586-8059-e9a92f148860",
    "num_chave_viagem": "3522073318365800013358000021009283122234751",
    "num_viagem": "2473",
    "num_serie_viagem": "0",
    "dhr_envio": "2026-07-16T17:16:39-03:00",
    "data_envio": "2026-07-16",
    "hora_envio": "17:16:39",
    "data_emissao": "2022-07-12",
    "hora_emissao": "13:00",
    "valor": 48062.31,
    "tipo_documento": "58",
    "tipo_emitente": "1",
    "cod_documento_emitente": "32004241000103",
    "nom_razao_social_emitente": "SUGGEST ONE CONSULTORIA LTDA",
    "cod_documento_relacionamento": "32004241000103",
    "nom_razao_social_relacionamento": "SUGGEST ONE CONSULTORIA LTDA",
    "cod_documento_parceiro": "61074175000138",
    "nom_razao_social_parceiro": "MAPFRE SEGUROS GERAIS S.A.",
    "cod_documento_cliente": "03724425000131",
    "nom_razao_social_cliente": "Vanessa Regina Alves",
    "qcte": "6",
    "qnfe": 0,
    "qmdfe": 0,
    "tomadores": [ { "cnpj_tomador": "16922038000151" } ],
    "descarregamento": [
      {
        "ibge_descarga": "3502804",
        "municipio_descarga": "ARACATUBA",
        "sigla_uf": "SP",
        "chave": "35220706016175000173570000002363391135461970"
      }
    ],
    "documentos": [
      { "tipo": "57 - Conhecimento", "chave": "35220706016175000173570000002363391135461970" }
    ]
  }
}
```

Campos

Identificação da viagem

Campo	Tipo	Descrição
<code>id_averbgo_mdfe</code>	string (UUID)	Identificador do processamento no AverbGo — grave este valor
<code>num_chave_viagem</code>	string (44)	Chave de acesso do MDF-e
<code>num_viagem</code>	string	Número do manifesto
<code>numSerie_viagem</code>	string	Série
<code>tipo_documento</code>	number string	<code>58</code>
<code>tipo_emitente</code>	string	Tipo do emitente

Datas e valor

Campo	Tipo	Descrição
<code>dhr_envio</code>	string ISO 8601	Data/hora do processamento, com offset
<code>data_envio</code> / <code>hora_envio</code>	string	Mesma informação, em campos separados
<code>data_emissao</code> / <code>hora_emissao</code>	string	Emissão do MDF-e
<code>data_embarque</code> / <code>hora_embarque</code>	string	Embarque, quando informado (pode vir vazio)
<code>valor</code>	number	Valor total da carga manifestada

Partes envolvidas

Campo	Descrição
<code>cod_documento_emitente</code> / <code>nom_razao_social_emitente</code>	Emitente do MDF-e
<code>cod_documento_relacionamento</code> / <code>nom_razao_social_relacionamento</code>	Relacionamento (segurado)
<code>cod_documento_parceiro</code> / <code>nom_razao_social_parceiro</code>	Seguradora/parceiro
<code>cod_documento_cliente</code> / <code>nom_razao_social_cliente</code>	Cliente
<code>tomadores[].cnpj_tomador</code>	Tomadores do serviço

Conteúdo do manifesto

Campo	Tipo	Descrição
<code>qcte</code> / <code>qnfe</code> / <code>qmdfe</code>	number string	Quantidade de CT-e, NF-e e MDF-e relacionados
<code>documentos[]</code>	array	Documentos vinculados: <code>tipo</code> (ex.: <code>57 - Conhecimento</code>) e <code>chave</code>
<code>descarregamento[]</code>	array	Municípios de descarga: <code>ibge_descarga</code> , <code>municipio_descarga</code> , <code>sigla_uf</code> , <code>chave</code>
<code>carregamento[]</code>	array	Municípios de carregamento
<code>motoristas[]</code> / <code>veiculos[]</code>	array	Motoristas e veículos do manifesto
<code>naver</code>	—	Informação de averbação relacionada
<code>tip_status</code> / <code>des_tipo_status</code>	number/string	Situação do manifesto
<code>des_endereco_xml</code>	string	Referência interna do XML armazenado

``qcte`` e ``qnfe`` com tipos diferentes

Nos retornos observados, `qcte` chega como string (`"6"`) e `qnfe` / `qmdfe` como número (`0`). Converta antes de comparar.

Cancelamento e encerramento

Os eventos de MDF-e **não** usam este bloco: são devolvidos em `event_mdfe`. Ver [Eventos](#).

11 Eventos (`event` e `event_mdfe`)

Eventos são operações sobre um documento **já enviado**: cancelamento de CT-e, NF-e e Outros, e cancelamento ou encerramento de MDF-e.

A API usa **dois blocos diferentes**, com estruturas bastante distintas:

Documento do evento	Bloco	Formato
CT-e, NF-e, Outros	<code>event</code>	array de eventos, com os campos já normalizados
MDF-e	<code>event_mdfe</code>	objeto que encapsula o XML do evento convertido

Trate os dois separadamente

Não escreva um único parser para "evento". Verifique primeiro qual bloco veio e leia os campos correspondentes.

Cancelamento de CT-e / NF-e / Outros (`event`)

```
{
  "message": "Evento de cancelamento processado com sucesso",
  "event": [
    {
      "id_documento_cancelado": "6818e4e8-a392-42f9-ba02-013cc7ed2dd7",
      "id_documento_afetado": "25fabdaf-236f-42b7-97cd-dd8b420fda95",
      "chave_documento": "53220834274233001257550000018220451121035004",
      "tipo_evento": "110111",
      "num_protocolo": "143140038502164",
      "data_envio": "2026-07-16",
      "hora_envio": "16:42:42",
      "data_cancelamento_sefaz": "2023-11-03",
      "hora_cancelamento_sefaz": "17:31:28",
      "tip_status": 1,
      "des_tipo_status": "Ativo",
      "cod_documento_emitente": "33183658000135",
      "nom_razao_social_emitente": "NR PARTICIPACOES LTDA",
      "cod_documento_relacionamento": "33183658000135",
      "nom_razao_social_relacionamento": "NR PARTICIPACOES LTDA",
      "cod_documento_parceiro": "61074175000138",
      "nom_razao_social_parceiro": "MAPFRE SEGUROS GERAIS S.A.",
      "cod_documento_cliente": "03724425000131",
      "nom_razao_social_cliente": "GPS CORRETAGENS DE SEGUROS LTDA",
      "xml_evento": "<procEventoCTe versao=\"3.00\" ...>"
    }
  ]
}
```

Campos

Campo	Tipo	Descrição
<code>id_documento_cancelado</code>	string (UUID)	Identificador do evento de cancelamento registrado
<code>id_documento_afetado</code>	string (UUID)	Identificador da averbação que foi cancelada
<code>chave_documento</code>	string (44)	Chave de acesso do documento cancelado
<code>tipo_evento</code>	string number	<code>110111</code> = cancelamento
<code>num_protocolo</code>	string	Protocolo do evento na SEFAZ
<code>data_envio</code> / <code>hora_envio</code>	string	Data e hora do processamento no AverbGo
<code>data_cancelamento_sefaz</code> / <code>hora_cancelamento_sefaz</code>	string	Data e hora do cancelamento na SEFAZ
<code>tip_status</code> / <code>des_tipo_status</code>	number/string	Situação do registro do evento
<code>cod_documento_*</code> / <code>nom_razao_social_*</code>	string	Emitente, relacionamento, parceiro e cliente
<code>xml_evento</code>	string	Eco do XML do evento enviado

``event`` é um array

Mesmo com um único evento, o bloco vem como lista. Percorra o array — não leia `event.id_documento_cancelado` diretamente.

``event`` também aparece em erros — como objeto

Em respostas de erro, a chave `event` pode conter um **objeto** de validação (`{ "errors": [...], "errorCount": 1, "stage": "validation", ... }`), e não a lista de eventos processados. Confirme o HTTP e verifique `Array.isArray(event)` antes de ler. Ver [Erros](#).

Cancelamento e encerramento de MDF-e (event_mdfe)

```
{
  "event_mdfe": {
    "produto": "TRANSPORTE",
    "avercacaoMeio": 2,
    "id_pessoa_emitente": "...",
    "cod_usuario": "NR PARTICIPACOES",
    "proceventomdfe": {
      "eventomdfe": {
        "infevento": {
          "cnpj": "33183658000135",
          "chmdfe": "53220834274233001257550000018220451121035007",
          "dhevento": "2022-07-05T15:09:05-04:00",
          "tpevento": "110111",
          "nsequevento": "1",
          "detevento": {
            "evcancmdfe": {
              "descevento": "Cancelamento",
              "nprot": "950220004478454",
              "xjust": "feito para teste"
            }
          }
        }
      }
    },
    "reteventomdfe": { "...": "retorno da SEFAZ" }
  },
  "des_endereco_xml": "event-dfe-f48be3b0-0197-4844-b434-65b422efa9a6"
}
```

Campos

Campo	Descrição
<code>produto</code>	Produto que processou o evento (ex.: <code>TRANSPORTE</code>)
<code>avercacaoMeio</code>	Meio de averbação utilizado
<code>id_pessoa_emitente</code>	Emitente no AverbGo
<code>cod_usuario</code>	Identificação do emitente
<code>proceventomdfe.eventomdfe.infevento.chmdfe</code>	Chave do MDF-e afetado
<code>proceventomdfe.eventomdfe.infevento.tpevento</code>	<code>110111</code> cancelamento · <code>110112</code> encerramento
<code>proceventomdfe.eventomdfe.infevento.dhevento</code>	Data/hora do evento
<code>proceventomdfe.eventomdfe.infevento.nsequevento</code>	Sequência do evento
<code>...detevento.evcancmdfe.nprot</code>	Protocolo do cancelamento
<code>...detevento.evcancmdfe.xjust</code>	Justificativa do cancelamento
<code>...detevento.evencmdfe.nprot</code>	Protocolo do encerramento
<code>...detevento.evencmdfe.dtenc</code>	Data do encerramento
<code>...detevento.evencmdfe.cuf</code> / <code>cmun</code>	UF e município do encerramento
<code>proceventomdfe.reteventomdfe</code>	Retorno da SEFAZ, conforme o layout oficial
<code>des_endereco_xml</code>	Referência interna do XML armazenado (fica fora de <code>event_mdfe</code>)

Estrutura espelha o XML da SEFAZ

Diferente do bloco `event` , aqui os dados **não são normalizados**: o conteúdo é o próprio evento MDF-e convertido de XML para JSON, com as tags em minúsculas (`infevento` , `tpevento` , `detevento`). Campos como `data_envio` , `hora_envio` , `id_documento_cancelamento` ou `tipo_evento` **não existem** neste bloco — o tipo do evento está em `proceventomdfe.eventomdfe.infevento.tpevento` .

Distinguir cancelamento de encerramento

```
tpevento == "110111" → Cancelamento de MDF-e
tpevento == "110112" → Encerramento de MDF-e
```

O bloco de detalhe também muda: `evcancmdfe` para cancelamento e `evencmdfe` para encerramento.

Pré-requisito

O documento original precisa ter sido enviado e processado antes do evento. Cancelamentos de documentos desconhecidos são recusados com `EVENT_PROCESSING_ERROR` e mensagens como "O documento não pertence ao emitente".

Evento repetido

Reenviar um evento já processado devolve `success: true` com mensagem "Evento de Cancelamento já processado anteriormente" e `data` vazio (`{}`). É idempotente — trate como sucesso.

12 Erros

Códigos HTTP

HTTP	Significado	Ação recomendada
200 / 201 / 202	Processado — verifique o corpo , pode haver falha por produto	Consolidar produtos
400	Requisição malformada (XML inválido, header ausente)	Corrigir — não reenviar igual
401	Chave inválida, desativada ou emitente inativo	Parar a fila do cliente e avisar o usuário
404	Recurso não encontrado (ex.: apólice inexistente)	Não reenviar sem correção
422	Regra de negócio impediu a averbação	Não reenviar sem correção
500	Falha no processamento	Reenviar com backoff
503 / 504	Serviço indisponível ou lento	Reenviar com backoff

HTTP 2xx não significa "averbado"

Um documento pode voltar com HTTP 200 e conter produtos com `success: false`, ou um `produtos` vazio. Sempre inspecione o corpo.

Formatos de corpo em erro

A API usa quatro formatos. Implemente a leitura na ordem abaixo.

1. Somente `message`

```
{ "message": "Data de Embarque deve ser maior ou igual à Data de Emissão." }
```

2. `message` + `error` (texto)

```
{
  "message": "Documento ja existe na base de dados. Chave do documento: 3522...",
  "error": "Documento ja existe na base de dados. Chave do documento: 3522..."
}
```

3. `message` + `event` com validações

```
{
  "message": "Usuário inativo ou não encontrado",
  "event": {
    "errors": [
      {
        "field": "User",
        "message": "Usuário inativo ou não encontrado",
        "rule": "USUARIO_INATIVO_OU_NAO_ENCONTRADO",
        "context": {}
      }
    ],
    "errorCount": 1,
    "stage": "validation",
    "rule": "USUARIO_INATIVO_OU_NAO_ENCONTRADO",
    "businessRuleViolation": true
  }
}
```

Campo	Descrição
<code>event.errors[].field</code>	Campo/entidade que falhou
<code>event.errors[].message</code>	Descrição do problema
<code>event.errors[].rule</code>	Código da regra violada — use este valor na lógica
<code>event.errorCount</code>	Quantidade de erros
<code>event.stage</code>	Etapa (ex.: <code>validation</code>)
<code>event.businessRuleViolation</code>	<code>true</code> quando é regra de negócio (não adianta reenviar)

4. `message` + `produtos` com o detalhe por produto

```
{
  "message": "TRANSPORTE: ERRO - O Prazo para averbação foi ultrapassado. Prazo de Averbação: Horas após Emissão | RCV: ERRO - Nenhuma apolice RCV vigente encontrada",
  "produtos": {
    "TRANSPORTE": {
      "success": false,
      "statusCode": 422,
      "message": "O Prazo para averbação foi ultrapassado. Prazo de Averbação: Horas após Emissão",
      "data": null,
      "error": "AVERBACAO_RECUSADA"
    },
    "RCV": {
      "success": false,
      "statusCode": 404,
      "message": "Nenhuma apolice RCV vigente encontrada",
      "error": "APOLICE_NAO_ENCONTRADA"
    }
  }
}
```

Leia o `produtos` também nas falhas

Mesmo em resposta de erro, `produtos` pode trazer o motivo **de cada produto**. Guardar apenas a `message` do topo perde essa informação — que é justamente a que o usuário precisa para corrigir.

Leitura recomendada da mensagem

```
mensagem = produtos[*].message (quando houver, uma por produto)
ou event.errors[*].message
ou error
ou message
ou "Falha na comunicação com o AverbGo"
```

Códigos de `error` conhecidos

Código	Ocorre quando	Reenviar resolve?
<code>AVERBACAO_RECUSADA</code>	Regra da apólice impediu a averbação (prazo ultrapassado, limite extrapolado)	Não — corrigir a origem
<code>APOLICE_NAO_ENCONTRADA</code>	Não há apólice vigente para o produto/emitente	Não — verificar contratação
<code>DOCUMENTO_DUPLICADO</code>	Documento já processado. Vem com <code>success: true</code>	Não é erro
<code>MDFE_PROCESSING_ERROR</code>	Falha ao processar dados do MDF-e (ex.: registro de veículo)	Às vezes — corrigir dados
<code>EVENT_PROCESSING_ERROR</code>	Falha ao processar o evento (documento não pertence ao emitente, viagem inexistente)	Não — verificar o documento original
<code>USUARIO_INATIVO_OU_NAO_ENCONTRADO</code>	Chave de emitente inativo	Não — regularizar no portal

Mensagens frequentes

Mensagem	Causa provável
Data de Embarque deve ser maior ou igual à Data de Emissão.	Tag extra <code>embarque</code> anterior à emissão do documento
O Prazo para averbação foi ultrapassado. Prazo de Averbação: ...	Envio fora do prazo da apólice
Documento já existe na base de dados. Chave do documento: ...	Reenvio de documento já averbado
Emitente sem meio de averbação habilitado para os produtos do documento	Emitente sem produto habilitado
Nenhuma apólice RCV vigente encontrada	RC-V sem apólice vigente
Tipo de documento 'CTE' não autorizado para apólice RCV. Tipos permitidos: ...	Documento fora dos tipos aceitos pela apólice
O documento não pertence ao emitente	Evento enviado com chave de outro emitente
Usuário inativo ou não encontrado	Chave revogada ou emitente inativo

O que reenviar

Classe	Exemplos	Política
Transitório	<code>500</code> , <code>503</code> , <code>504</code> , timeout, falha de rede	Reenviar com backoff exponencial (ex.: 1 min, 5 min, 15 min, máx. 5 tentativas)
Permanente	<code>400</code> , <code>401</code> , <code>404</code> , <code>422</code> , <code>businessRuleViolation: true</code>	Não reenviar automaticamente — exige correção ou ação do usuário
Idempotente	<code>DOCUMENTO_DUPLICADO</code> , evento já processado	Reenvio é seguro e não duplica averbação

Não faça retentativa infinita

Documentos com recusa de regra de negócio não mudam de resultado por repetição. Reenvio em laço só gera carga e atrasa os documentos válidos da fila.

13 Guia de implementação

Recomendações para uma integração estável em produção.

Tratamento completo da resposta

Pseudocódigo cobrindo todos os formatos descritos neste manual:

```

function tratarRetorno(httpStatus, corpo) {
  if (httpStatus === 401) {
    return { resultado: 'CHAVE_INVALIDA', pararFila: true };
  }
  if (httpStatus >= 400) {
    return {
      resultado: 'FALHA',
      mensagem: extrairMensagem(corpo),
      reterar: [500, 502, 503, 504].includes(httpStatus),
    };
  }

  // 1) multi-hierarquia tem precedência
  const produtos = Array.isArray(corpo.emitentes)
    ? corpo.emitentes.flatMap(e => Object.entries(e.produtos ?? {}))
    : Object.entries(corpo.produtos ?? {});

  // 2) consolidação por produto
  if (produtos.length > 0) {
    const ok = produtos.filter(([, p]) => p.success === true);
    const falha = produtos.filter(([, p]) => p.success === false);
    const antts = extrairAntts(corpo);

    if (ok.length === 0) return { resultado: 'FALHA', detalhes: falha };
    if (falha.length > 0) return { resultado: 'PARCIAL', detalhes: falha, antts };
    return { resultado: 'AVERBADO', antts };
  }

  // 3) resposta simples, sem produtos
  if (corpo.endorsement) {
    return { resultado: 'AVERBADO', antts: extrairAntts(corpo) };
  }
  if (corpo.mdfe) {
    return { resultado: 'AVERBADO', protocolo: corpo.mdfe.id_averbgo_mdfe };
  }
  if (Array.isArray(corpo.event)) {
    const evento = corpo.event[0];
    return { resultado: 'EVENTO_OK', protocolo: evento?.id_documento_cancelado };
  }
  if (corpo.event_mdfe) {
    const info = corpo.event_mdfe.proceventomdfe?.eventomdfe?.infoevento;
    return { resultado: 'EVENTO_OK', tipo: info?.tpevento };
  }

  // 4) nada reconhecido
  return {
    resultado: 'FALHA',
    mensagem: corpo.message ?? 'Retorno não reconhecido',
  };
}

function extrairAntts(corpo) {
  const coletar = produtos => Object.values(produtos ?? {})
    .map(p => p?.data?.endorsement)
    .filter(Boolean);

  const lista = [];
  if (Array.isArray(corpo.emitentes)) {
    corpo.emitentes.forEach(e => lista.push(...coletar(e.produtos)));
  } else {
    lista.push(...coletar(corpo.produtos));
  }
}

```

```

}
if (corpo.endorsement) lista.push(corpo.endorsement);

return lista.flatMap(e => (e.antts ?? []).map(a => ({
  antt: a.antt,
  apolice: a.numero_apolice,
  dataAverbacao: e.dhr_averbacao,
})));
}

```

Regras de ouro

1. **Decida por** `success`, nunca pela `message`. O texto pode mudar sem aviso.
2. **Percorra coleções:** `emitentes`, `produtos`, `antts` e `event` são listas ou mapas — nunca apenas o primeiro item.
3. **Trate campos opcionais:** `data`, `data.endorsement` e `error` podem não existir.
4. **Não falhe com campos desconhecidos:** novos campos podem ser incluídos a qualquer momento; o parse precisa ignorá-los.
5. **Converta tipos antes de comparar:** alguns campos numéricos chegam como texto e vice-versa (ver avisos abaixo).
6. **Guarde o JSON bruto do retorno**, ao menos por alguns meses. É a evidência da averbação em caso de sinistro ou divergência.

Fila de envio

- Envie **um documento por requisição**.
- Use uma fila com concorrência limitada (2 a 5 requisições simultâneas por cliente é suficiente na maioria das operações).
- Aplique timeout generoso: a resposta devolve o XML original e pode passar de 100 KB. **60 segundos** é um valor seguro.
- Persista o estado de cada documento (enviado, averbado, parcial, erro) antes de seguir para o próximo.

Retentativa

Situação	Reenviar?
Timeout, <code>500</code> , <code>503</code> , <code>504</code> , erro de rede	Sim, com backoff exponencial e limite de tentativas
<code>400</code> , <code>404</code> , <code>422</code> , <code>businessRuleViolation: true</code>	Não — exige correção
<code>401</code>	Não — interromper a fila do cliente e avisar o usuário
Sucesso parcial	Somente após corrigir a causa do produto que falhou

O reenvio é **idempotente**: um documento já averbado retorna `DOCUMENTO_DUPLICADO` com `success: true`, sem gerar nova averbação. Em caso de dúvida sobre o resultado (por exemplo, timeout depois do envio), reenviar é seguro.

Ordem dos envios

1. documento autorizado (CT-e / NF-e / MDF-e)
2. ...somente depois: eventos daquele documento (cancelamento, encerramento)

Um evento de documento que a API não conhece é recusado.

O que exibir ao usuário

Resultado	Sugestão de exibição
Averbado	Número ANTT + data/hora da averbação
Averbado (MDF-e)	<code>id_averbgo_mdfe</code> como protocolo
Sucesso parcial	Destaque visual próprio, listando qual produto falhou e por quê
Falha	Mensagem do produto (ou <code>event.errors[].message</code>), com ação sugerida
Chave inválida	Aviso para o responsável verificar a chave no portal

Sucesso parcial precisa de visibilidade

Se a interface tratar parcial como sucesso, o usuário nunca perceberá que um produto ficou sem averbação. Use um estado próprio na listagem e nos relatórios.

Avisos consolidados

Pontos observados no comportamento atual da API que exigem atenção no código:

#	Comportamento	Como se proteger
1	<code>tipo_documento</code> chega ora como número (<code>57</code>), ora como string (<code>"57"</code>)	Compare com <code>String(valor)</code>
2	<code>qcte</code> chega como string e <code>qnfe</code> / <code>qmdfe</code> como número	Converta antes de comparar
3	<code>event</code> é array no sucesso e objeto de validação em erros	Verifique <code>Array.isArray()</code>
4	<code>event_mdfe</code> não tem campos normalizados — espelha o XML da SEFAZ	Leia <code>proceventomdfe.eventomdfe.infevento.tpevento</code>
5	<code>produtos</code> pode vir <code>{}</code> vazio com mensagem de impedimento	<code>{}</code> = falha, não sucesso
6	<code>data</code> pode ser objeto, array ou ausente conforme produto e operação	Verifique o tipo antes de acessar
7	O prefixo <code>/v1</code> existe só em produção	Use constante por ambiente
8	A resposta ecoa o XML enviado em <code>dfe.xml</code>	Descarte antes de persistir, se não for usar

Checklist de homologação

- ☐ Envio de CT-e autorizado — ANTT gravado
- ☐ Envio de NF-e autorizada — ANTT gravado
- ☐ Envio de MDF-e — `id_averbgo_mdfe` gravado
- ☐ Cancelamento de CT-e/NF-e — evento processado
- ☐ Cancelamento e encerramento de MDF-e — evento processado
- ☐ Emitente com mais de uma hierarquia — todas as hierarquias tratadas
- ☐ Emitente com dois produtos — sucesso parcial exibido corretamente
- ☐ Documento fora do prazo — recusa tratada sem retentativa infinita
- ☐ Reenvio de documento já averbado — tratado como duplicidade
- ☐ Chave inválida (401) — fila interrompida e usuário avisado
- ☐ Timeout — retentativa com backoff, sem duplicar averbação
- ☐ Retorno bruto armazenado

14 Exemplos por cenário

Respostas reais (com identificadores mascarados) e o tratamento esperado.

1. Averbação de CT-e — sucesso

```
{
  "message": "Processo de Averbação concluído com sucesso!",
  "endorsement": {
    "id_documento": "ec4c67bb-95f3-4118-a0d2-93c32a9fd851",
    "tipo_documento": "57",
    "dhr_averbacao": "2026-07-03T17:34:33-03:00",
    "des_tipo_status": "Averbado",
    "antts": [ { "antt": "0572007272419599200011857001000004465000", "numero_apolice": "5500032923" } ]
  }
}
```

Tratamento: documento averbado. Gravar `antts[0].antt` e `dhr_averbacao`.

2. Emitente com dois produtos — tudo processado

```
{
  "message": "Processo de Averbação concluído com sucesso!",
  "produtos": {
    "TRANSPORTE": { "success": true, "statusCode": 201, "data": { "endorsement": { "antts": [ {
      "antt": "0012301283..." } ] } } },
    "RCV": { "success": true, "statusCode": 202, "data": { "processed": true, "pending": true } }
  }
}
```

Tratamento: averbado. O ANTT vem do `TRANSPORTE`; o RC-V está processado com conclusão assíncrona (`pending: true`).

3. Sucesso parcial

```
{
  "message": "Falha ao criar registro de Veículo",
  "produtos": {
    "TRANSPORTE": { "success": false, "statusCode": 500, "error": "MDFE_PROCESSING_ERROR",
      "message": "Falha ao criar registro de Veículo" },
    "RCV": { "success": true, "statusCode": 202, "message": "Documento RC-V processado" }
  }
}
```

Tratamento: estado **parcial**. Exibir que o Transporte não foi averbado e o motivo. Reenviar apenas após corrigir o cadastro do veículo.

4. Multi-hierarquia — averbação

```
{
  "message": "2 emitente(s) processado(s) com sucesso",
  "emitentes": [
    {
      "emitente": "NR PARTICIPACOES", "cliente": "GPS-PAMCARY", "parceiro": "MAPFRE",
      "produtos": {
        "TRANSPORTE": {
          "success": true, "statusCode": 201,
          "data": {
            "endorsement": {
              "tipo_documento": "57", "dhr_averbacao": "2026-07-16T16:48:13-03:00",
              "antts": [ { "antt": "0012301283318365800013557000000035006034" } ]
            }
          }
        }
      }
    },
    {
      "emitente": "NR PARTICIPACOES", "cliente": "DATASYSTEM - RCV", "parceiro": "DATASYSTEM",
      "produtos": {
        "RCV": {
          "success": true, "statusCode": 202,
          "data": {
            "processed": true, "pending": true
          }
        }
      }
    }
  ]
}
```

Tratamento: averbado. É o mesmo emitente em duas hierarquias — gravar o ANTT **de cada hierarquia**, porque cada uma responde a uma apólice diferente.

5. Multi-hierarquia — cancelamento com falha parcial

```
{
  "message": "[NR PARTICIPACOES] TRANSPORTE: Evento de cancelamento processado com sucesso | [NR PARTICIPACOES] RCV: ERRO - O documento não pertence ao emitente",
  "emitentes": [
    {
      "produtos": {
        "TRANSPORTE": {
          "success": true, "statusCode": 200,
          "data": {
            "event": [ {
              "id_documento_cancelado": "6818e4e8-...", "tipo_evento": "110111",
              "data_envio": "2026-07-16", "hora_envio": "16:42:42"
            } ]
          }
        }
      }
    },
    {
      "produtos": {
        "RCV": {
          "success": false, "statusCode": 500, "error": "EVENT_PROCESSING_ERROR",
          "message": "O documento não pertence ao emitente - ..."
        }
      }
    }
  ]
}
```

Tratamento: estado **parcial**. O cancelamento valeu para o Transporte; o RC-V precisa de verificação — o documento original provavelmente não foi enviado para aquele relacionamento.

6. MDF-e processada

```
{
  "message": "Processamento de MDFe concluído com sucesso!",
  "mdfe": {
    "id_averbgo_mdfe": "1075df34-b408-4586-8059-e9a92f148860",
    "num_chave_viagem": "3522073318365800013358000021009283122234751",
    "dhr_envio": "2026-07-16T17:16:39-03:00",
    "qcte": "6", "qnfe": 0
  }
}
```

Tratamento: processado. Não há ANTT — gravar `id_averbgo_mdfe` como protocolo.

7. Encerramento de MDF-e

```
{
  "emitentes": [ { "produtos": { "TRANSPORTE": { "success": true, "statusCode": 200,
    "data": { "event_mdfe": { "proceventomdfe": { "eventomdfe": { "infevento": {
      "chmdfe": "5322083427423300125755000001822045112103500**",
      "tpevento": "110112",
      "detevento": { "evencmdfe": { "descevento": "Encerramento",
        "nprot": "9432200125484**", "dtenc": "2022-07-05" } } }
      } } } } } } } ]
}
```

Tratamento: encerramento confirmado (`tpevento` `110112`). Protocolo em `evencmdfe.nprot` .

8. Recusa por prazo

```
{
  "message": "TRANSPORTE: ERRO - O Prazo para averbação foi ultrapassado. Prazo de Averbação: Horas após Emissão | RCV: ERRO - Nenhuma apolice RCV vigente encontrada",
  "produtos": {
    "TRANSPORTE": { "success": false, "statusCode": 422, "error": "AVERBACAO_RECUSADA",
      "message": "O Prazo para averbação foi ultrapassado. Prazo de Averbação: Horas após Emissão" },
    "RCV": { "success": false, "statusCode": 404, "error": "APOLICE_NAO_ENCONTRADA",
      "message": "Nenhuma apolice RCV vigente encontrada" }
  }
}
```

Tratamento: falha (nenhum produto com sucesso). Não reenviar automaticamente: exibir os dois motivos ao usuário.

9. Documento já averbado

```
{
  "message": "Documento ja existe na base de dados. Chave do documento: 3522...",
  "error": "Documento ja existe na base de dados. Chave do documento: 3522..."
}
```

Tratamento: duplicidade. Se o TMS não tem registro da averbação anterior, consulte o portal ou o suporte para recuperar o ANTT.

10. Chave de emitente inativo

```
{
  "message": "Usuário inativo ou não encontrado",
  "event": {
    "errors": [ { "field": "User", "message": "Usuário inativo ou não encontrado",
                  "rule": "USUARIO_INATIVO_OU_NAO_ENCONTRADO" } ],
    "errorCount": 1, "stage": "validation", "businessRuleViolation": true
  }
}
```

Tratamento: interromper a fila daquele cliente e orientar a verificação da chave no portal AverbGo.

15 Suporte e FAQ

Contato

Assunto	Canal
Dúvidas de integração, homologação, liberação de acesso	sac@averbgo.com.br
Portal do segurado (chaves, apólices, consultas)	averbgo.com.br
Geração de chave de acesso	averbgo.com.br/main/chaves

Ao abrir um chamado sobre um documento específico, informe:

- CNPJ do emitente;
- chave de acesso do documento (44 dígitos);
- data e hora do envio;
- **o JSON de retorno recebido** (é o que permite identificar o processamento).

Perguntas frequentes

Posso enviar o XML antes da autorização da SEFAZ? Não. O documento precisa estar autorizado e protocolado.

Preciso enviar o MDF-e se já envio os CT-e? Sim, quando o segurado tem o produto correspondente. O MDF-e representa a viagem e complementa a informação das cargas.

O reenvio do mesmo documento gera averbação duplicada? Não. A API identifica o documento pela chave e responde `DOCUMENTO_DUPLICADO` com `success: true`.

Recebi HTTP 200, mas um produto veio com `success: false`. O documento está averbado? Parcialmente. O produto que falhou **não** foi averbado. Trate como pendência.

Como sei qual é o número da averbação? É o `antt`, em `antts[].antt`. Pode haver mais de um quando o emitente tem mais de uma apólice vigente.

O que faço quando recebo 401? Interrompa os envios daquele cliente e verifique a chave no portal. Reenviar não resolve.

Qual o tempo de resposta esperado? Varia com o tamanho do documento e a quantidade de produtos. Use timeout de 60 segundos.

Existe limite de requisições? Não há limite publicado, mas mantenha a concorrência moderada (2 a 5 envios simultâneos por cliente) para não formar fila desnecessária.

Perdi o retorno de um envio. Como recupero o ANTT? Consulte o portal AverbGo ou o suporte, informando a chave do documento.

A API tem consulta de status por documento? Não faz parte deste manual. Se a sua operação precisa disso, fale com o suporte.

Histórico deste manual

Versão	Data	Alterações
2.0	Julho/2026	Reescrita completa. Inclusão dos formatos <code>emitentes</code> , <code>produtos</code> , <code>mdfe</code> , <code>event</code> e <code>event_mdfe</code> ; catálogo de erros e códigos; guia de implementação; exemplos por cenário; separação de ambientes e autenticação.
1.0	2022	Versão inicial: envio de XML, tags extras e retorno <code>endorsement</code> .